

new_assignmentDF&SQL

February 24, 2026

```
[1]: from pyspark.sql import SparkSession
spark=SparkSession.builder.appName("New Assignment").getOrCreate()

sc=spark.sparkContext
```

DATAFRAMES

```
[2]: from pyspark.sql.functions import col,sum,avg,lit,split,element_at

cust_df=spark.read.csv("cust.csv",header=True,inferSchema=True)
prod_df=spark.read.csv("prod.csv",header=True,inferSchema=True)
ord_df=spark.read.csv("ord.csv",header=True,inferSchema=True)
```

```
[3]: # 1. Display the details of premium customers who are from "Houston" or
      ↳ "Chicago".

cust_df.filter( # here instead of col("ctype") ==> cust_df.ctype can also be
      ↳ used
      (col("ctype") == "Premium")
      & (
          (col("city") == "Houston") | (col("city") == "Chicago")
      ) # (col("city").isin("Houston,Chicago"))
    ).show()
```

```
+-----+-----+-----+-----+-----+
|cust_id|      name|  ctype|      email|   city|
+-----+-----+-----+-----+-----+
|    103| Peter Jones|Premium| p.jones@email.com|Chicago|
|    105|David Wilson|Premium|dave.w@business.com|Houston|
+-----+-----+-----+-----+-----+
```

```
[5]: # 2. Display customer id, name, and city of the customers whose last name
      ↳ starts with a lowercase character.

# rlike , & split
# split(col("name")," ").getItem(-1).rlike("^[a-z]") # negative indexing is
      ↳ not supported in getItem()
```

```

from pyspark.sql.functions import col, split, size

cust_df.filter(
    split(col("name"), " ").getItem(size(split(col("name"), " ") - 1).
    ↪rlike("[a-z]"))
).select("cust_id", "name", "city").show()

# using udf
# from pyspark.sql.functions import udf, col
# from pyspark.sql.types import BooleanType

# def check_lower(name):
#     return name.split(" ")[-1][0].islower()

# check_udf = udf(check_lower, BooleanType())

# cust_df.filter(
#     check_udf(col("name"))
# ).select("cust_id", "name", "city").show()

```

c:\Users\emada\AppData\Local\Programs\Python\Python311\Lib\site-packages\pyspark\sql\classic\column.py:359: FutureWarning: A column as 'key' in getItem is deprecated as of Spark 3.0, and will not be supported in the future release. Use `column[key]` or `column.key` syntax instead.

```
warnings.warn(

+-----+-----+-----+
|cust_id|    name|    city|
+-----+-----+-----+
|    101| John doe| New York|
|    102|Jane smith|Los Angeles|
+-----+-----+-----+
```

[6]: # 3. Display total number of customers based on customer type in reverse,
 ↪alphabetical order of customer type.

```

from pyspark.sql.functions import *
cust_df.groupBy("ctype").agg(count("*").alias("Total_Customers")).
    ↪orderBy(col("ctype").desc()).show()

```

```

+-----+-----+
|  ctype|Total_Customers|
+-----+-----+
|Regular|                2|
|Premium|                3|
+-----+-----+
```

```
[7]: # 4. Display the total number of products based category and subcategory in
      ↪alphabetical order of category and subcategory.
prod_df.groupBy("category", "subcategory").agg(count("*").alias("Total_
      ↪Products")).orderBy(col("category"), col("subcategory")).show()

# or
# prod_df.groupBy("category", "subcategory").count().
      ↪orderBy("category", "subcategory").show()
```

category	subcategory	Total Products
Apparel	Footwear	2
Apparel	Tops	1
Electronics	Accessories	1
Electronics	Laptops	2
Electronics	Televisions	1
Home Goods	Kitchen Appliances	1

```
[ ]: # 5. Display the details of product(s) with lowest price.

min_price=prod_df.agg(min("price")).collect()[0][0] # .collect() --> converts_
      ↪dataframe to list, in that first row[0], first column[0]
# or
# min_price = prod_df.agg(min("price").alias("min_price")).first()["min_price"]
prod_df.filter(col("price")==min_price).show()
```

p_id	pname	category	subcategory	price
1003	Wireless Mouse	Electronics	Accessories	25.0
1008	T Shirt	Apparel	Tops	25.0

Situation	Use
Need single value	.first() or .collect()
Comparing inside filter	Must compare with scalar
Comparing two DataFrames	Use join()

```
[51]: # 6. Display the details of the orders with highest price.
max_price=ord_df.agg(max("ord_amount")).collect()[0][0]
ord_df.filter(col("ord_amount")==max_price).show()
```

o_id	cust_id	p_id	ord_date	del_date	quantity	ord_amount
2001	101	1001	2025-08-11	2025-08-14	1	1200.0
2006	101	1001	2025-08-16	2025-08-19	1	1200.0

[13]: # 7. Display the details of the products whose price is more than the average price of the product available.

```
avg_price=prod_df.agg(avg("price")).collect()[0][0]
prod_df.filter(col("price")>avg_price).show()
```

p_id	pname	category	subcategory	price
1001	Laptop Pro	Electronics	Laptops	1200.0
1002	Smart TV	Electronics	Televisions	850.5
1006	MacBook Air	Electronics	Laptops	1199.0

[14]: # 8. Display name of the products and number of times orders placed for them.

```
ord_df.groupBy("p_id").count().join(prod_df,"p_id").select("pname","count").show()
```

pname	count
Running Shoes	2
T Shirt	1
Smart TV	2
Laptop Pro	2
Wireless Mouse	2
Coffee Maker	1

[17]: # 9. Display the details of customers who are yet to place an order.

```
cust_df.join(ord_df,"cust_id","left_anti").show()
# or
cust_df.join(ord_df, "cust_id", "left").filter(col("o_id").isNull()).select(
    cust_df["*"]
).show()
```

cust_id	name	ctype	email	city
103	Peter Jones	Premium	p.jones@email.com	Chicago

	105	David Wilson	Premium	dave.w@business.com	Houston
--	-----	--------------	---------	---------------------	---------

cust_id	name	ctype	email	city
103	Peter Jones	Premium	p.jones@email.com	Chicago
105	David Wilson	Premium	dave.w@business.com	Houston

```
[19]: # 10. Display product name and total revenue generated by each product.
ord_df.join(prod_df, "p_id").groupBy("pname").sum("ord_amount").show()
# or
ord_df.join(prod_df, "p_id").groupBy("pname").agg(sum("ord_amount").alias("Total_
Revenue")).show()
```

pname	sum(ord_amount)
Wireless Mouse	229.95
T Shirt	75.0
Coffee Maker	75.0
Running Shoes	299.85
Smart TV	1701.0
Laptop Pro	2400.0

pname	Total Revenue
Wireless Mouse	229.95
T Shirt	75.0
Coffee Maker	75.0
Running Shoes	299.85
Smart TV	1701.0
Laptop Pro	2400.0

```
[56]: # 11. Display the o_id, name, p_name, ord date, quantity for each order in
ascending order of oid.
ord_df.join(cust_df, "cust_id").join(prod_df, "p_id").select(
    "o_id", "name", "pname", "ord_date", "quantity"
).orderBy("o_id").show()
```

o_id	name	pname	ord_date	quantity
------	------	-------	----------	----------

2001	John doe	Laptop Pro	2025-08-11	1
2002	Jane smith	Smart TV	2025-08-12	1
2003	Mary Brown	Running Shoes	2025-08-13	2
2004	John doe	Wireless Mouse	2025-08-14	5
2005	Jane smith	Coffee Maker	2025-08-15	1
2006	John doe	Laptop Pro	2025-08-16	1
2007	Mary Brown	T Shirt	2025-08-17	3
2008	Jane smith	Wireless Mouse	2025-08-18	2
2009	Mary Brown	Smart TV	2025-08-19	1
2010	John doe	Running Shoes	2025-08-20	1

+-----+-----+-----+-----+-----+

```
[ ]: # 12. Display p_id, p name and total revenue generated as "totalRevenue" by
      ↳ each product in the ascending order of p_id and p name. Display totalRevenue
      ↳ as if any product is yet to generate revenue.
from pyspark.sql.functions import sum, coalesce, lit

prod_df.join(ord_df, "p_id", "left").groupBy("p_id", "pname").agg(
    coalesce(sum("ord_amount"), lit(0)).alias("totalRevenue")
).orderBy("p_id", "pname").show()

# or
prod_df.join(ord_df, "p_id", "left").groupBy("p_id", "pname").agg(
    sum("ord_amount").alias("totalRevenue")
).na.fill(0, ["totalRevenue"]).orderBy("p_id", "pname").show() # for nulls take
↳ totalRevenue as 0
```

p_id	pname	totalRevenue
1001	Laptop Pro	2400.0
1002	Smart TV	1701.0
1003	Wireless Mouse	229.95
1004	Coffee Maker	75.0
1005	Running Shoes	299.85
1006	MacBook Air	0.0
1007	Hiking Boots	0.0
1008	T Shirt	75.0

+-----+-----+-----+

p_id	pname	totalRevenue
1001	Laptop Pro	2400.0
1002	Smart TV	1701.0
1003	Wireless Mouse	229.95
1004	Coffee Maker	75.0

1005	Running Shoes	299.85
1006	MacBook Air	0.0
1007	Hiking Boots	0.0
1008	T Shirt	75.0

```
+-----+-----+-----+
```

```
[58]: # 13. Display cust id, firstname and last name for the customers who are not
      ↪from "Houston" or "Chicago"
```

```
cust_df.filter(~col("city").isin("Houston", "Chicago")).select(
    "cust_id",
    split(col("name"), " ").getItem(0).alias("firstname"),
    split(col("name"), " ").getItem(1).alias("lastname"),
).show()
```

```
+-----+-----+-----+
|cust_id|firstname|lastname|
+-----+-----+-----+
|    101|    John|    doe|
|    102|    Jane|   smith|
+-----+-----+-----+
```

```
[ ]: # 14. Display the o id, cust_id, ord_date, del date, delivery days for each
      ↪product.
```

```
from pyspark.sql.functions import datediff

ord_df.select(
    "o_id",
    "cust_id",
    "ord_date",
    "del_date",
    datediff("del_date", "ord_date").alias("delivery_days"), #
    ↪datediff(end_date, start_date)
).show()
```

```
+-----+-----+-----+-----+-----+
|o_id|cust_id| ord_date| del_date|delivery_days|
+-----+-----+-----+-----+-----+
|2001|    101|2025-08-11|2025-08-14|          3|
|2002|    102|2025-08-12|2025-08-15|          3|
|2003|    104|2025-08-13|2025-08-16|          3|
|2004|    101|2025-08-14|2025-08-17|          3|
|2005|    102|2025-08-15|2025-08-18|          3|
|2006|    101|2025-08-16|2025-08-19|          3|
|2007|    104|2025-08-17|2025-08-20|          3|
|2008|    102|2025-08-18|2025-08-21|          3|
```

2009	104	2025-08-19	2025-08-22	3
2010	101	2025-08-20	2025-08-23	3

+-----+

```
[62]: # 15. Display the o id, name of the customer, name of the product and day took
      ↪for delivering the product as delivery days for all orders.
      from pyspark.sql.functions import datediff

      ord_df.join(cust_df, "cust_id").join(prod_df, "p_id").select(
          "o_id", "name", "pname", datediff("del_date", "ord_date").
          ↪alias("delivery_days")
      ).show()
```

o_id	name	pname	delivery_days
------	------	-------	---------------

+-----+

2001	John doe	Laptop Pro	3
2002	Jane smith	Smart TV	3
2003	Mary Brown	Running Shoes	3
2004	John doe	Wireless Mouse	3
2005	Jane smith	Coffee Maker	3
2006	John doe	Laptop Pro	3
2007	Mary Brown	T Shirt	3
2008	Jane smith	Wireless Mouse	3
2009	Mary Brown	Smart TV	3
2010	John doe	Running Shoes	3

+-----+

```
[63]: # 16. The delivery for the "Electronics" are delayed by 5 more days, display
      ↪the o id, cust id, pid, expected delivery date for the Electronics product.
      from pyspark.sql.functions import date_add

      ord_df.join(prod_df, "p_id").filter(col("category") == "Electronics").select(
          "o_id", "cust_id", "p_id", date_add("del_date", 5).
          ↪alias("expected_delivery_date")
      ).show()
```

o_id	cust_id	p_id	expected_delivery_date
------	---------	------	------------------------

+-----+

2001	101	1001	2025-08-19
2002	102	1002	2025-08-20
2004	101	1003	2025-08-22
2006	101	1001	2025-08-24
2008	102	1003	2025-08-26
2009	104	1002	2025-08-27


```
+---+-----+---+-----+

```

SQL Answers

```
[22]: orders=ord_df.createOrReplaceTempView("orders")
customers=cust_df.createOrReplaceTempView("customers")
products=prod_df.createOrReplaceTempView("products")
```

```
[ ]: # 1. Premium customers from houston or chicago
spark.sql(
    "select * from customers where ctype='Premium' and city in_
    ↳('Houston','Chicago')"
).show()
```

```
+-----+-----+---+-----+
|cust_id|      name|  ctype|      email|  city|
+-----+-----+---+-----+
|    103| Peter Jones|Premium| p.jones@email.com|Chicago|
|    105|David Wilson|Premium|dave.w@business.com|Houston|
+-----+-----+---+-----+
```

Operator	Supports	Use Case
LIKE	%, _	Simple matching
RLIKE	Full regex	Pattern / ranges / advanced

```
[ ]: # 2. cust_id, name, city where last name starts with lowercase
spark.sql(
    """
    select cust_id,city,name from customers
        where split(name,' ')[1] rlike '^[a-z]'
    """
).show()
```

```
+-----+-----+---+-----+
|cust_id|      city|      name|
+-----+-----+---+-----+
|    101|   New York| John doe|
|    102|Los Angeles|Jane smith|
+-----+-----+---+-----+
```

```
[27]: # 3. Total customers based on customer type (reverse alphabetical)
spark.sql(
    """
    select ctype,count(*) as total_customers
    from customers
```

```

        group by ctype
        order by ctype desc
        """
    ).show()

```

```

+-----+-----+
|  ctype|total_customers|
+-----+-----+
|Regular|                2|
|Premium|              3|
+-----+-----+

```

```

[28]: # 4. Total products based on category & subcategory
spark.sql("""
select category,subcategory,count(*) as total_products
      from products group by category,subcategory
      order by category,subcategory
      """).show()

```

```

+-----+-----+-----+
|  category|      subcategory|total_products|
+-----+-----+-----+
|   Apparel|      Footwear|            2|
|   Apparel|         Tops|            1|
|Electronics|   Accessories|            1|
|Electronics|      Laptops|            2|
|Electronics|  Televisions|            1|
|Home Goods|Kitchen Appliances|            1|
+-----+-----+-----+

```

```

[29]: # 5. Products with lowest price
spark.sql("""
select * from products
      where price=(select min(price) from products)
      """).show()

```

```

+---+-----+-----+-----+-----+
|p_id|      pname|  category|subcategory|price|
+---+-----+-----+-----+-----+
|1003|Wireless Mouse|Electronics|Accessories| 25.0|
|1008|      T Shirt|   Apparel|      Tops| 25.0|
+---+-----+-----+-----+-----+

```

```

[ ]: # 6. Orders with highest order amount
spark.sql("""
select * from orders

```

```

        where ord_amount==(select max(ord_amount) from orders)
    """).show()

```

o_id	cust_id	p_id	ord_date	del_date	quantity	ord_amount
2001	101	1001	2025-08-11	2025-08-14	1	1200.0
2006	101	1001	2025-08-16	2025-08-19	1	1200.0

```

[31]: # 7. Products whose price > average price
spark.sql("""
select * from products
        where price>(select avg(price) from products)
""").show()

```

p_id	pname	category	subcategory	price
1001	Laptop Pro	Electronics	Laptops	1200.0
1002	Smart TV	Electronics	Televisions	850.5
1006	MacBook Air	Electronics	Laptops	1199.0

```

[32]: # 8. Product name & number of times ordered
spark.sql("""
select p.pname,count(*) as total_orders
        from orders o
        join products p
        on o.p_id=p.p_id
        group by p.pname
""").show()

```

pname	total_orders
Wireless Mouse	2
T Shirt	1
Coffee Maker	1
Running Shoes	2
Smart TV	2
Laptop Pro	2

[34]: # 9. Customers who are yet to place an order

```
spark.sql(  
    """  
    SELECT *  
    FROM customers c  
    LEFT JOIN orders o  
    ON c.cust_id = o.cust_id  
    WHERE o.cust_id IS NULL  
    """  
) .show()
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+  
+-----+-----+-----+-----+  
|cust_id|      name|  ctype|      email|  
city|o_id|cust_id|p_id|ord_date|del_date|quantity|ord_amount|  
+-----+-----+-----+-----+-----+-----+-----+-----+  
+-----+-----+-----+-----+  
|    103| Peter Jones|Premium| p.jones@email.com|Chicago|NULL|    NULL|NULL|  
NULL|    NULL|    NULL|    NULL|  
|    105|David Wilson|Premium|dave.w@business.com|Houston|NULL|    NULL|NULL|  
NULL|    NULL|    NULL|    NULL|  
+-----+-----+-----+-----+-----+-----+-----+-----+  
+-----+-----+-----+-----+  

```

[]: # -- 9. Customers who are yet to place an order

```
# SELECT *  
# FROM customers c  
# LEFT JOIN orders o  
# ON c.cust_id = o.cust_id  
# WHERE o.cust_id IS NULL;
```

-- 10. Product name and total revenue generated by each product

```
# SELECT p.pname,  
#       SUM(o.ord_amount) AS total_revenue  
# FROM orders o  
# JOIN products p  
# ON o.p_id = p.p_id  
# GROUP BY p.pname;
```

```

# -- 11. o_id, customer name, product name, ord_date, quantity (ascending order
↳of o_id)

# SELECT o.o_id,
#         c.name,
#         p.pname,
#         o.ord_date,
#         o.quantity
# FROM orders o
# JOIN customers c
# ON o.cust_id = c.cust_id
# JOIN products p
# ON o.p_id = p.p_id
# ORDER BY o.o_id;

# -- 12. p_id, pname and totalRevenue (0 if product has no sales)

# SELECT p.p_id,
#         p.pname,
#         COALESCE(SUM(o.ord_amount), 0) AS totalRevenue
# FROM products p
# LEFT JOIN orders o
# ON p.p_id = o.p_id
# GROUP BY p.p_id, p.pname
# ORDER BY p.p_id, p.pname;

# -- 13. cust_id, firstname, lastname not from Houston or Chicago

# SELECT cust_id,
#         SPLIT(name, ' ')[0] AS firstname,
#         SPLIT(name, ' ')[1] AS lastname
# FROM customers
# WHERE city NOT IN ('Houston', 'Chicago');

# -- 14. o_id, cust_id, ord_date, del_date, delivery_days

# SELECT o_id,
#         cust_id,
#         ord_date,
#         del_date,
#         DATEDIFF(del_date, ord_date) AS delivery_days

```

```

# FROM orders;

# -- 15. o_id, customer name, product name, delivery_days

# SELECT o.o_id,
#        c.name,
#        p.pname,
#        DATEDIFF(o.del_date, o.ord_date) AS delivery_days
# FROM orders o
# JOIN customers c
# ON o.cust_id = c.cust_id
# JOIN products p
# ON o.p_id = p.p_id;

# -- 16. Electronics delivery delayed by 5 days (expected delivery date)

# SELECT o.o_id,
#        o.cust_id,
#        o.p_id,
#        DATE_ADD(o.del_date, 5) AS expected_delivery_date
# FROM orders o
# JOIN products p
# ON o.p_id = p.p_id
# WHERE p.category = 'Electronics';

```

```

[35]: with open("queries.txt","r") as f:
      queries=f.read().split(";")

```

```

for q in queries:
    if q.strip()!="":
        spark.sql(q).show()

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|cust_id|      name|  ctype|      email|
city|o_id|cust_id|p_id|ord_date|del_date|quantity|ord_amount|
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|   103| Peter Jones|Premium| p.jones@email.com|Chicago|NULL|  NULL|NULL|
NULL|   NULL|   NULL|   NULL|
|   105|David Wilson|Premium|dave.w@business.com|Houston|NULL|  NULL|NULL|
NULL|   NULL|   NULL|   NULL|
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+

```

pname	total_revenue
Wireless Mouse	229.95
T Shirt	75.0
Coffee Maker	75.0
Running Shoes	299.85
Smart TV	1701.0
Laptop Pro	2400.0

o_id	name	pname	ord_date	quantity
2001	John doe	Laptop Pro	2025-08-11	1
2002	Jane smith	Smart TV	2025-08-12	1
2003	Mary Brown	Running Shoes	2025-08-13	2
2004	John doe	Wireless Mouse	2025-08-14	5
2005	Jane smith	Coffee Maker	2025-08-15	1
2006	John doe	Laptop Pro	2025-08-16	1
2007	Mary Brown	T Shirt	2025-08-17	3
2008	Jane smith	Wireless Mouse	2025-08-18	2
2009	Mary Brown	Smart TV	2025-08-19	1
2010	John doe	Running Shoes	2025-08-20	1

p_id	pname	totalRevenue
1001	Laptop Pro	2400.0
1002	Smart TV	1701.0
1003	Wireless Mouse	229.95
1004	Coffee Maker	75.0
1005	Running Shoes	299.85
1006	MacBook Air	0.0
1007	Hiking Boots	0.0
1008	T Shirt	75.0

cust_id	firstname	lastname
101	John	doe
102	Jane	smith

o_id	cust_id	ord_date	del_date	delivery_days
2001	101	2025-08-11	2025-08-14	3
2002	102	2025-08-12	2025-08-15	3
2003	104	2025-08-13	2025-08-16	3
2004	101	2025-08-14	2025-08-17	3
2005	102	2025-08-15	2025-08-18	3
2006	101	2025-08-16	2025-08-19	3
2007	104	2025-08-17	2025-08-20	3
2008	102	2025-08-18	2025-08-21	3
2009	104	2025-08-19	2025-08-22	3
2010	101	2025-08-20	2025-08-23	3

o_id	name	pname	delivery_days
2001	John doe	Laptop Pro	3
2002	Jane smith	Smart TV	3
2003	Mary Brown	Running Shoes	3
2004	John doe	Wireless Mouse	3
2005	Jane smith	Coffee Maker	3
2006	John doe	Laptop Pro	3
2007	Mary Brown	T Shirt	3
2008	Jane smith	Wireless Mouse	3
2009	Mary Brown	Smart TV	3
2010	John doe	Running Shoes	3

o_id	cust_id	p_id	expected_delivery_date
2001	101	1001	2025-08-19
2002	102	1002	2025-08-20
2004	101	1003	2025-08-22
2006	101	1001	2025-08-24
2008	102	1003	2025-08-26
2009	104	1002	2025-08-27